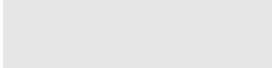


Program Assignment 2

(SPRING 2026)



目錄

摘要.....	1
---------	---

課程：演算法概論

組別：第八組

組員：人工智慧三 A 411203712 陳郁潔

人工智慧三 A 411221003 楊雅羽

人工智慧三 A 411211943 吳佳彥

人工智慧三 A 411211969 黃燦凱

一、 情境、系統與問題定義.....	2
1-1 情景描述.....	2
1-2 範例地圖.....	2
1-3 頂點資料與邊資料.....	3
1-4 系統設計.....	4

1-5 資料模型與使用流程.....	4
1-5.1 資料模型	4
1-5.2 使用流程	5
1-6 系統功能需求.....	6
1-7 使用者介面設計.....	10
1-8 問題定義.....	10
1-8.1 單源最短路徑問題(Single-Source Shortest Path,SSSP)...	11
1-8.2 最短配送計畫問題(TSP 變形).....	11
1-8.3 系統限制條件	12
1-8.4 最佳化優化目標	13
1-9 AI 工具協助項目	13
二、 提出的演算法.....	13
2-1 Dijkstra 最短路徑演算法	13
2-2 TSP 分支定界法(Branch-and-Bound).....	14
2-3 演算假設.....	15

2-4 設計考慮因素.....	15
2-5 AI 工具協助項目	15
三、 實作與結果.....	16
3-1 程式實作概述.....	16
3-2 主要資料結構.....	16
3-3 核心搜尋函式.....	16
3-4 測試案例與輸出結果.....	16
3-5 案例手算驗證.....	16
3-6 三組測試結果比較.....	16
3-7 實作章節的 AI 工具協助項目.....	16
四、 討論.....	16
4-1 系統優點.....	16
4-2 系統限制.....	17
4-3 結論與反思.....	17
4-3.1 系統成果	17

4-3.2 未來改進的方向	17
----------------------------	-----------

4-3.3 遇到的挑戰	18
--------------------------	-----------

摘要

根據作業 PDF 上的要求，設計一套配送計畫資訊系統，系統環境以加權圖 $G=(V,E)$ 模擬都市路徑網路，頂點代表城市或配送據點，邊權重則代表車輛行駛時間，同時將車輛載重、車隊數量，以及派送貨取貨複合式任務情境納入環境限制考量。

在系統設計上，我們不限定地圖樣式，使用者能夠自行創建地圖，包含地圖頂點及路徑，在計算最短路徑前，使用者能夠選擇指定的出發點及需要配送經過的地點，系統會根據使用者的選擇去計算最短路徑。

實作部分我們使用的是課堂教過的 Dijkstra 演算法，與 TSP 分支定界法 (Branch-and-Bound)，使用 Dijkstra 演算法找到單源最短路徑，用 TSP 分支定界法找最佳配送順序。

一、 情境、系統與問題定義

1-1 情描述景

系統背景是快遞公司員工需要幫司機規劃配送計畫，希望能夠加快配送速度，在最短時間內完成任務、加速物件送達，實際規劃時需要考量到公司所擁有的車輛總數量、每輛車的最大載運容量限制與車輛在兩點之間移動時間。

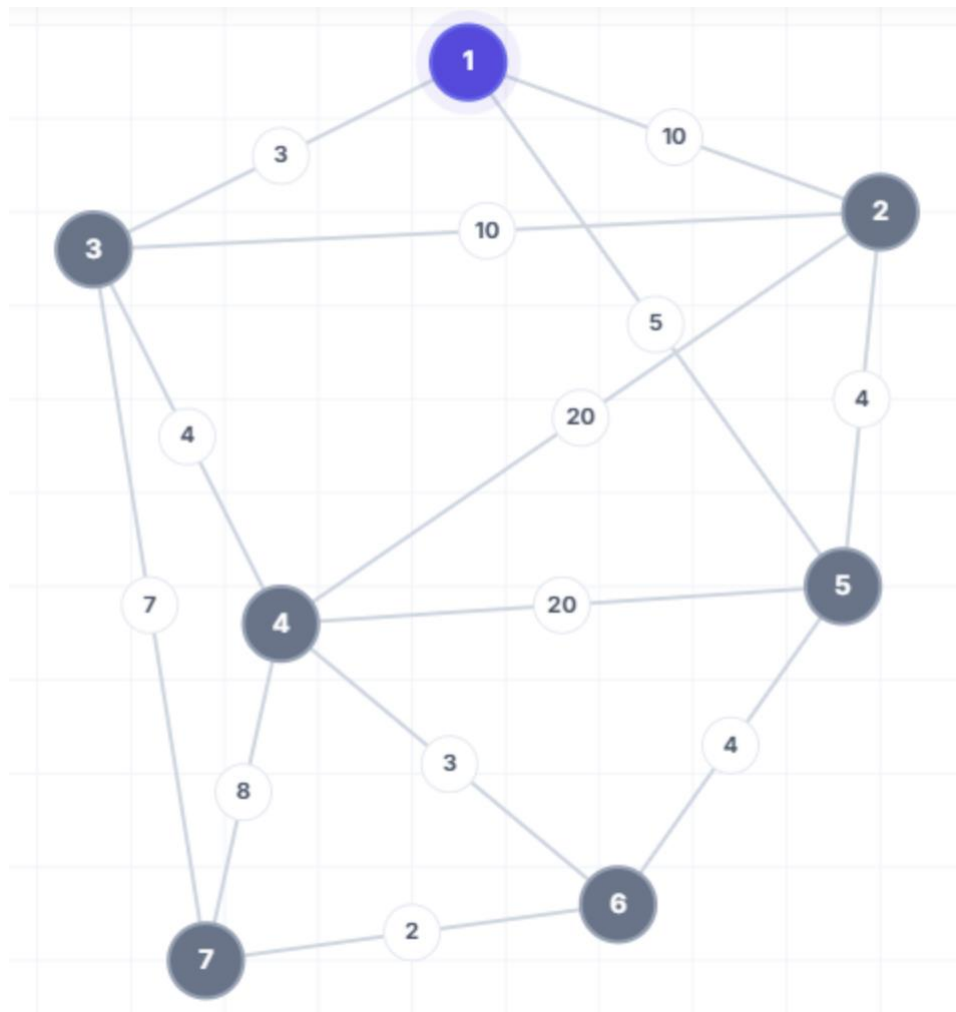
系統將城市交通網路以帶權無向圖 $G=(V, E)$ 來表示，頂點表示城市或配送據點，邊代表兩城市或配送據點間的連接道路，邊權重代表車輛在城市(配送據點)與城市(配送據點)之間行駛所需的時間

範例地圖中，各個頂點皆能雙向通行，地圖為無向量圖

- V (頂點集合): 代表城市或配送據點。每個頂點 $v \in V$ 代表一個具體的地理位置 (如城市、倉庫、客戶地點)。
- E (邊集合): 代表兩個城市之間的道路連接。邊 $(i,j) \in E$ 表示城市(配送據點) i 與城市(配送據點) j 之間存在直接的交通路線。
- $w: E \rightarrow R^+$ (邊權重函數): $w(i,j)$ 表示從城市(配送據點) i 到城市(配送據點) j 的行駛時間。由於道路是雙向的, $w(i,j) = w(j,i)$ 。

1-2 範例地圖

圖一表示七個節點的配送地圖，呈現各個城市(配送據點)之間的聯通關係，及頂點、邊與邊權重，為後續系統設計與演算法測試的基礎。



圖一

1-3 頂點資料與邊資料

範例地圖中共有七個頂點，表示共有七個城市(配送據點)， $V=\{1, 2, 3, 4, 5, 6, 7\}$ ，圖中共有 13 條無向量的邊，邊上的數值代表兩站點之間的行駛時

間，表 1 為各個城市(配送據點)之間的城市地圖鄰接矩陣（行駛時間），例：城市(配送據點)1 到城市(配送據點)3 的直接行駛時間為 3，對角線為 0，無法直達的點以 ∞ 表示。

	城市 1	城市 2	城市 3	城市 4	城市 5	城市 6	城市 7
城市 1	0	10	3	∞	5	∞	∞
城市 2	10	0	10	20	4	∞	∞
城市 3	3	10	0	4	∞	∞	7
城市 4	∞	20	4	0	20	3	8
城市 5	5	4	∞	20	0	4	∞
城市 6	∞	∞	∞	3	4	0	2
城市 7	∞	∞	7	8	∞	2	0

表 1 城市地圖鄰接矩陣（行駛時間）

1-4 系統設計

系統是一套為物流快遞公司打造的配送計畫資訊系統，目標是提供一個直觀、高效的數位化平台，讓使用者自由輸入/匯入都市地圖（節點與路段時

間)，並在輸入出發點與複數個派送貨、取貨目的地後，自動產生最短配送時間的路徑規劃(最佳配送路徑規劃)。

系統提供直覺的圖形化操作介面，支援滑鼠拖曳、滾輪縮放與格線輔助的城市地圖視覺化區域，並配置包含選取、新增點線、刪除及載入範例等功能的地圖編輯工具列，讓使用者能自由建立任意地圖，使用者可透過控制面板彈性設定起點、勾選多個目的地並使用最短路徑（Dijkstra）與路線規劃（分支定界法）演算法，計算結果除在結果展示區詳細呈現最短時間、配送順序與各段實際路徑外，更會透過動畫演示在地圖上動態模擬配送車輛的實際行駛路線。

1-5 資料模型與使用流程

1-5.1 資料模型

1-5.2 使用流程

以下表格為使用範例地圖的使用流程

步驟	使用者操作 / 互動	系統回應
1	使用者進入系統，點擊 頂部工具列的載入範例 快捷按鈕	系統於視覺化區域渲染 出基礎都市地圖（含節 點與道路），背景同步顯

		示輔助格線，並進入就緒狀態
2	使用者移至側邊的控制面板： 1. 於下拉選單選擇一個起始城市（起點 s）。 2. 勾選多個欲造訪的配送目的地	系統實時鎖定起點與目標節點集合，並在 Canvas 地圖上將這些被選取的節點進行高亮顯示，以利使用者核對
3	使用者在控制面板中，依據評估需求勾選： 1. 最短路徑演算法（Dijkstra） 2. 路線規劃演算法（分支定界法）	系統將前端選擇的演算法參數封裝為請求物件，傳遞給後端路徑優化計算引擎，做好運算準備
4	使用者按下控制面板底部的計算最短配送計畫	後端引擎動態載入地圖資料，依序運行指定演算法，排列組合出總耗

		時最小的閉合配送迴圈 路徑
5	使用者查看介面下方的 結果展示區	系統輸出並顯示：總最 短配送時間、配送路線 順序清單，以及拆解後 各路段的詳細實際行駛 路徑與時間明細
6	使用者觀看 Canvas 畫 布上的動態變化	地圖非相關路段自動變 暗，系統以動畫方式 (如移動的小貨車圖示 或動態流線)，沿著計算 出的最佳路徑順序進行 動態行駛模擬演示
7	若發生道路突發狀況， 使用者在畫布上點擊特 定道路，直接修改兩點 間的配送時間，並重新 點擊計算	系統實時修改資料庫中 的邊權重 $w(i,j)$ ，核心引 擎重新動態規劃，於結 果區與畫布上即時更新 更換後的最短路線

1-6 系統功能需求

系統需求可以分成功能性需求與非功能性需求。功能性需求說明系統必須做什麼(表 2)；非功能需求定義了系統運作時的品質屬性、效能指標、技術限制與使用者體驗(表三)。

代號	功能需求	說明
FR-1.1	地圖視覺化	系統利用 HTML5 Canvas 即時繪製都市網路之節點（城市/配送據點）與道路邊緣（路段）
FR-1.2	畫布動態操作	視覺化畫布支援滑鼠拖曳平移與滑鼠滾輪縮放，且縮放時文字與點線不可失真
FR-1.3	拓撲編輯工具	系統頂部工具列提供選取（拖曳節點）、新增城市、新增道路與刪除四種核心編輯工具，供使用者自由建立任意地圖

FR-1.4	路段時間權重輸入	當使用者新增或點選道路時，系統彈出輸入框，允許使用者手動輸入或修改兩點間的配送行駛時間（邊權重 $w(i,j)$ ）
FR-1.5	快速快取管理	工具列提供載入範例快捷鍵（自動渲染出題目 Fig. 1 的 7 點地圖）與全部清除功能
FR-2.1	起始點指定	控制面板提供選取機制（下拉選單，供使用者指定單一起始城市 s ）
FR-2.2	目的地多選	控制面板多選功能，允許使用者指定複數個配送目的地 $\{v_1, v_2, \dots, v_k\}$
FR-3.1	關鍵數據輸出	計算完成後，結果展示區以顯著字體呈現「總最短配送時間」
FR-3.2	配送順序明細	結果展示區必須清楚條列出排列組合優化後的最佳造訪順序清單（例： $1 \rightarrow 5 \rightarrow 7 \rightarrow 1$ ）

FR-3.3	實際路徑拆解	當兩造訪點之間無法直達而需繞行其餘未勾選節點時，系統詳細拆解並條列出該段的實際行駛軌跡與個別耗時
FR-4.1	路徑動畫演示	點擊計算後，系統在 Canvas 地圖上以動畫方式沿著規劃路徑動態模擬車輛的實際行駛路線
FR-4.2	背景視覺聚焦	動畫播放期間，系統自動淡化非相關的道路與節點，以突顯當前的最佳配送路線

表 2 功能需求表

代號	非功能需求	說明
NFR-1.1	易用性	1. 系統介面佈局需符合直覺，使用者從編輯/確認地圖、設定起終點參數到產出路徑的核心工作流，點擊次數控制在 3 次 Click 內完成

		2. 支援滑鼠拖曳與滾輪縮放的 HTML5 Canvas 畫布，需提供防呆與容錯提示（如未選起點時點擊計算，系統需跳出輕量化警告警示，而非程式崩潰）
NFR-1.2	可解釋性	1. 系統不能僅給出最終結果，必須在結果展示區詳細拆解並還原計算邏輯。例如：當兩目的地之間無法直達而需繞行其餘未勾選節點時，系統必須條列出該段的實際行駛軌跡與個別耗時 2. 透過前端 Canvas 視覺化動畫演示配送車輛的實際行駛路線，讓使用者能直觀理解演算法排程的合理性與先後順序
NFR-1.3	可維護性	1. 系統架構採用前後端完全解耦設計。前端 UI 渲染邏輯（Canvas 畫布）與後端圖論計算引擎必須獨立運作，未來若要新增、替換其餘演算法，不可影響前端介面架構

		2. 後端核心圖形資料結構模組化封裝，以便未來能無縫擴充納入車輛載重容量限制與複合任務類型的約束條件
NFR-1.4	執行效率	1. 在地圖總節點數 $V \leq 20$ 且勾選目的地數量 $k \leq 10$ 的標準測試情境下，核心優化引擎（包含單源最短路徑與群組路徑優化）在 500 毫秒（0.5 秒）內完成計算並回傳結果 2. Canvas 視覺化區域在執行滑鼠拖曳平移、滾輪縮放以及車輛行駛動畫演示時，畫面更新率穩定維持在 60 FPS 以上，確保無肉眼可察覺的延遲

表 3 非功能需求表

1-7 使用者介面設計

系統提供直覺的圖形化操作介面，包含以下功能區域：

- 城市地圖視覺化區域：使用 HTML5 Canvas 即時繪製城市節點與道路邊緣，支援滑鼠拖曳平移和滾輪縮放。背景格線輔助使用者對齊節點位

置。

- 地圖編輯工具列：系統頂部提供完整的工具列，包含「選取（拖曳節點）」、「新增城市」、「新增道路」、「刪除」四種操作工具，以及「載入範例」和「全部清除」快捷按鈕。使用者可自由建立任意城市地圖。
- 控制面板：使用者可選擇起始城市、勾選多個配送目的地。
- 結果展示區：顯示最短配送時間、配送路線順序、各段的詳細實際路徑及時間。
- 動畫演示：計算完成後，系統會在地圖上以動畫方式展示配送車輛的實際行駛路線。

1-8 問題定義

系統的核心目標是為司機規劃出耗時最短的配送路徑。在實際的圖論與運籌學理論中，此目標須被拆解為以下兩個相互銜接的核心子問題：

1-8.1 單源最短路徑問題(Single-Source Shortest Path,SSSP)

在物流網路中，車輛往往無法在任意兩個客戶節點之間直達，必須透過都市的既有道路網路進行繞行，在規劃整體配送順序前，系統必須先求解起點與各節點間、以及各目的地兩兩之間的基礎最短旅行時間。

- 環境模型定義：給定一個連通的帶權無向圖 $G = (V, E)$ ，其中 V 為城市中所有站點與客戶地址的頂點集合， E 為道路邊緣集合。定義權重函數 $w: E \rightarrow R^+$ ，其中 $w(i,j)$ 代表車輛行駛於路段 (i,j) 所需的實際時間。

- 形式化定義：

輸入：帶權無向圖 $G = (V, E, w)$ ，指定之單一源點（起始站） $s \in V$

輸出：一個距離映射向量 $d: V \rightarrow R^+$ ，使得對於圖中任意節點 $v \in$

V ， $d(s, v)$ 皆代表從 s 到 v 所有可行路徑中總權重之最小值：

$$d(s, v) = \min \left\{ \sum_{e \in P} w(e) \mid P \text{ 是從 } s \text{ 到 } v \text{ 的一條路徑} \right\}$$

1-8.2 最短配送計畫問題(TSP 變形)

在求得任意兩點間的最短路徑成本後，系統需解決的核心挑戰是如何

在勾選的複數個目的地中，排列出一個總行駛時間最小化的巡迴順序，

此問題本質上屬於「旅行推銷員問題（TSP）」的變形。

- 形式化定義：
- 輸入：帶權無向圖 G 、確定的起始站點 $s \in V$ 、以及由使用者動態勾選的非空配送目的地子集合 $D = \{v_1, v_2, \dots, v_k\} \subseteq V$ （其中 k 為目的地總數，且 $s \notin D$ ）

- 輸出：一個一對一的排列映射函數 $\pi = \{1, 2, \dots, k\} \rightarrow \{1, 2, \dots, k\}$ ，定義車輛造訪目的地的先後順序，使得總配送計畫之時間成本（Cost）達到最小化：

$$\min_{\pi} \text{Cost}(\pi) = d(s, v_{\pi(1)}) + \sum_{i=1}^{k-1} d(v_{\pi(i)}, v_{\pi(i+1)}) + d(v_{\pi(k)}, s)$$

1-8.3 系統限制條件

演算法所搜尋出的路徑，必須同時且嚴格滿足以下限制，否則該路線即判定為不可行：

- 地圖連通性限制：輸入的都市地圖在數學上必須為連通圖。若圖中存在孤立節點，演算法將因計算出距離無限大（ ∞ ）而無法生成閉合配送路徑。
- 非負權重限制：實務上道路行駛時間必大於零（ $w(i, j) > 0$ ），系統禁止使用者輸入負數時間。
- 單一車輛巡迴限制：配送計畫必須嚴格遵守由同一個指定起點 s 出發，巡迴所有指定目的地後，最終返回原起點 s 的封閉迴圈（Hamiltonian Cycle）規範。
- 組合爆炸與目的地規模限制：最短配送計畫（TSP）屬於 NP-hard 問題。當選用精確解演算法（動態規劃法 / 分支定界法）時，時間複雜度呈指數或階乘成長。為滿足 500 毫秒內回應的效能需求，系統設定硬性防

呆：當目的地數量 $k > 15$ 時，將強制鎖定精確解，僅允許使用貪婪演算法求近似解

- 前端 Canvas 渲染極限：網頁端視覺化畫布在沒有特定圖形硬體加速的情境下，若地圖總節點數 $V > 500$ 且邊數 $E > 2000$ ，頻繁的縮放與平移重繪將無法穩定維持 60 FPS 的流暢度。
- 無向圖對稱性架構：目前底層數據結構基於無向圖設計，預設兩點間來回時間完全對稱 ($w(i,j) = w(j,i)$)，暫不支援單行道或特定時段調撥車道等非對稱路況。

1-8.4 最佳化優化目標

系統的核心最佳化優化目標旨在滿足所有限制條件的前提下，尋找最具效益的巡迴調度解，首要任務是將司機執行任務的總配送時間成本 (Total Time Cost) 最小化，透過底層點對點最短路徑演算法 (Dijkstra) 建構基礎距離矩陣，進而於多目的地排列組合中求解全局最優解 π^* ，使自指定起點 s 出發、巡迴所有造訪點並返回原起點之累計耗時函數 $f(\pi)$ 達到全局最小值；此外，系統在隱性指標上亦追求最小化實行路徑的空車繞行率以確保骨幹道路的行駛效益，並保留未來擴充多車負載平衡的多目標權重彈性；最後，在系統層面則著重於演算法搜尋效能的最佳化，透過高效的限界函數提升分支定界法的剪枝 (Pruning) 效率，最小化運算資源與迭代次數。

1-9 AI 工具協助項目

使用 AI 輔助工具 (Gemini) 來整理環境描述的文字結構。具體互動流程為：首先由作者提供作業題目描述與圖 1 的資料，再由 AI 協助將環境描述整理為結構化的形式定義，隨後對 AI 輸出進行人工校對與修正，確認所有數學符號與語義一致。

二、 提出的演算法

2-1 Dijkstra 最短路徑演算法

Dijkstra 演算法是解決單源最短路徑問題的經典貪婪演算法，適用於所有邊權重非負的圖。

- 演算法步驟：
 1. 初始化：將起點 s 的距離設為 0，其餘節點距離設為 ∞ 。建立未訪問節點集合 Q 。
 2. 從 Q 中選取距離最小的節點 u 。
 3. 對 u 的所有鄰居 v ，若 $\text{dist}[u] + w(u,v) < \text{dist}[v]$ ，則更新 $\text{dist}[v]$ 。
 4. 將 u 從 Q 中移除，重複步驟 2-3 直到 Q 為空
- 時間複雜度：

使用鄰接表 + 最小堆： $O((V + E) \log V)$ 。本系統使用簡單的陣列實作，複雜度為 $O(V^2)$ 。

2-2 TSP 分支定界法(Branch-and-Bound)

分支定界法是一種系統性搜尋所有可能解的方法，透過計算下界來修剪不可能產生最佳解的分支，從而提升效率。

- 演算法步驟：
 1. 以深度優先搜尋 (DFS) 的方式逐一嘗試訪問目的地的不同排列。
 2. 在每個搜尋節點，計算「當前已花費時間 + 下界估計」。
 3. 下界估計方法：對當前節點和每個未訪問節點，取其到任何其他未訪問節點（或起點）的最短邊之和。
 4. 若估計值 $\geq$ 目前已知最佳解，則修剪此分支 (Pruning)。
 5. 當所有目的地被訪問後，加上返回起點的時間，更新最佳解。
- 時間複雜度：

最壞情況： $O(k!)$ ，其中 k 為目的地數量。但透過修剪，實際執行時間通常遠小於此。

2-3 演算假設

- 地圖靜態性：演算法在執行計算期間，假設圖的拓撲結構（節點總數 V 、邊數 E ）及各路段的行駛時間 $w(i,j)$ 皆為靜態常數。
- 邊權重非負與物理合理性：系統假設所有道路的行駛時間成本必大於零
($\forall e \in E, w(e) > 0$)。
- 路網完全連通性：演算法假設輸入的都市地圖在數學上必須為連通圖。
- 數據對稱性與三角不等式：圖形架構基於無向圖設計，預設兩點間的往返行駛時間完全對稱 ($w(i,j) = w(j,i)$)。

2-4 設計考慮因素

在設計考慮因素上，主要著重算得快與畫面流暢兩大關鍵，首先，為了避免當送貨地點太多時，電腦因為計算量太大而卡死，系統設計了智慧切換機制，目的地在 15 個以內時，會使用能算出最完美路線的精確演算法；一旦超過 15 個，系統會自動改用反應最快的最近鄰居法來計算答案，確保網頁不崩潰，其次，在畫面呈現上，系統使用網頁畫布技術來繪製地圖，並將移動的小車動畫與靜態的背景地圖分開處理，讓使用者在放大、縮小或拖曳地圖時，畫面依然能保持流暢。最後，系統具備即時同步功能，只要使用者動態修改了某條路的行駛時間，舊的暫存資料就會立刻被刪除並重新計算，確保路徑清單、時間總計和小車跑的路線的正确性。

2-5 替代方法分析

單源最短路徑 (SSSP)與旅行推銷員問題 (TSP)的替代演算法

方法	核心概念	評估
Dijkstra 演算法 (本系統採用)	透過貪婪策略 (Greedy Method) 由起點不斷向外擴展， 每次選擇權重最小的鄰 接節點更新距離	優點：運算效率極高 ($O(V^2)$)，非常適合處理 現實稠密的物流路網 缺點：無法處理負權重 評估：物流的行駛時間絕對是正數，因此本系統採用此法能達到速度與精準度的最佳平衡
Bellman-Ford 演算法	以動態規劃為基礎，對地圖所有的邊進行多輪的鬆弛操作	優點：能完美處理包含「負權重」的邊，並能偵測圖中是否存在負迴圈 缺點：時間複雜度高 ($O(V \times E)$)

		評估： 物流系統不需要處理負權重，盲目使用此法會導致大規模地圖運算出現卡頓，故不採用
--	--	---

表 4 單源最短路徑 (SSSP)

方法	核心概念	評估
分支定界法 (B&B) (本系統採用)	透過深度優先 (DFS) 樹狀搜尋遍歷所有可能的配送組合。在搜尋過程中，利用**「下界函數 (LB)」預估剩餘未訪問節點的最小可能成本。若「當前累計成本 + 下界預估值」大於或等於當前全局最優解，則立即剪枝	時間複雜度： 最壞情況 $O(n!)$，但平均遠優於此 優點： 100% 保證找到絕對最省時的最佳解。下界優化後（如考慮未訪問最短出邊和），能極高效地剪去無效路徑

	(Pruning) **, 中斷該分支的搜尋	缺點： 面對極大規模節點 (如 $N > 20$) 仍有遭遇組合爆炸的風險 評估： 契合系統追求精確、絕對最優調度的核心邏輯，適合中小型物流路網
動態規劃法 (DP) (Held-Karp)	利用記憶化搜索 (Memoization), 將 TSP 拆解為重疊的子問題。其核心思想是：不論前面怎麼走，從當前城市 i 走完剩餘未訪問城市集合 S 回到起點的最短距離是固定的。系統會記錄已計算過的狀	時間複雜度： 嚴格限制在 $O(2^n \times n^2)$ 優點： 屬於精確解。在最壞情況下的時間表現比分支定界法更穩定，不會因圖形拓撲差而使速度劣化 缺點： 空間複雜度極高 (達 $O(2^n \times n)$)。非常消耗系統記憶體，且不論圖

	態 (城市 i, 集合 S), 避免重複計算	形好壞, 每次都必須算滿 固定複雜度, 缺乏彈性 評估: 雖然最壞情況時間 優於 B&B, 但在網頁前 端 (JavaScript 環境) 極 易因記憶體耗盡而崩潰, 故不採用
貪婪演算法 (Nearest Neighbor)	採用貪婪策略。物流小 車從起點出發, 每次都 直接選擇距離當前位置 最近 (行駛時間最短) 且尚未訪問的城市作為 下一個目的地, 直到所 有城市訪問完畢, 最後 直接連回起點	時間複雜度: 極低的 $O(n^2)$ 優點: 計算速度快到極 致, 程式碼極其簡單, 完 全不消耗運算資源與記憶 體 缺點: 無法保證最優解。 因為每一步都只看眼前利 益, 在接近尾聲時, 往往 會迫不得已必須連向大後

		方極遠的城市，導致路徑嚴重交錯、繞路（通常比最佳解差 25% 以上） 評估： 無法滿足智慧物流對成本控制的精確度要求，通常僅適合作為初始解生成，故不採用
--	--	--

表 5 旅行推銷員問題 (TSP)

2-6 AI 工具協助項目

在設計演算法時，使用了 AI 輔助工具來協助整理各演算法的步驟描述與複雜度分析。具體互動流程：作者先列出課堂所學的演算法清單（Dijkstra, Bellman-Ford, Branch-and-Bound, Greedy, DP），再由 AI 協助將虛擬碼整理為結構化的中文步驟描述，我們進行校對與調整。

三、實作與結果

3-1 程式實作概述

3-2 主要資料結構

3-3 核心搜尋函式

```
// =====  
// Algorithms: Dijkstra + Branch-and-Bound (FIXED)  
// =====  
function buildAdj(){const a={};nodes.forEach(n=>a[n.id]=[]);edges.forEach(e=>{a[e.source].push({t:e.target,w:e.weight});a[e.target].push({t:e.source,w:e.weight});});return a;}  
  
function dijkstra(startId){  
  const adj=buildAdj(),dist={},prev={},vis=new Set();  
  nodes.forEach(n=>{dist[n.id]=Infinity;prev[n.id]=null;});dist[startId]=0;  
  for(let i=0;i<nodes.length;i++){  
    let u=null,m=Infinity;for(const n of nodes){if(!vis.has(n.id)&&dist[n.id]<m){m=dist[n.id];u=n.id;}  
    if(u===null)break;vis.add(u);  
    for(const nb of(adj[u]||[])){const alt=dist[u]+nb.w;if(alt<dist[nb.t]){dist[nb.t]=alt;prev[nb.t]=u;}}  
  }  
  return(dist,prev);  
}  
  
function computeAllPairs(){const r=[];nodes.forEach(n=>{r[n.id]=dijkstra(n.id);});return r;}  
function getPath(u,v,ap){const p=[];let c=v;const pm=ap[u].prev;if(pm[c]===null&&u!==v)return[];while(c!==null){p.unshift(c);c=pm[c];}return p;}  
  
function tspBranchAndBound(startId,destIds,ap){  
  const n=destIds.length;let minCost=Infinity,bestPath=null,nodesExplored=0,pruneCount=0;  
  function lb(curr,unvis){  
    let b=0,m=ap[curr].dist[startId];  
    for(const u of unvis){m=Math.min(m,ap[curr].dist[u]);b+=m;}  
    for(const v of unvis){let m=ap[u].dist[startId];for(const v of unvis){if(v!==u)m=Math.min(m,ap[u].dist[v]);b+=m;}  
    return b;  
  }  
  function search(curr,vis,cost,path){  
    nodesExplored++;  
    if(vis.size===n){const t=cost+ap[curr].dist[startId];if(t<minCost){minCost=t;bestPath=[...path,startId];}return;}  
    const uv=destIds.filter(d=>!vis.has(d));  
    if(cost+lb(curr,uv)>=minCost){pruneCount++;return;}  
    for(const next of destIds){if(!vis.has(next)){vis.add(next);path.push(next);search(next,vis,cost+ap[curr].dist[next],path);vis.delete(next);path.pop();}}  
  }  
  search(startId,new Set(),0,[startId]);  
  return{cost:minCost,path:bestPath,nodesExplored,pruneCount};  
}
```

3-4 測試案例與輸出結果

3-5 案例手算驗證

3-6 三組測試結果比較

3-7 實作章節的 AI 工具協助項目

四、討論

4-1 系統優點

- 自由建圖：使用者可透過直覺的工具列自行建立、編輯城市地圖，不受限於固定的圖形結構。支援新增/刪除節點和邊、拖曳節點調整位置、雙擊修改邊權重等操作。
- 即時互動：系統提供即時的圖形化介面，使用者可直觀地看到城市地圖和最佳路線動畫。
- 零安裝部署：系統為純前端應用，只需瀏覽器即可使用，無需安裝任何軟體。
- 詳細路徑解析：系統不僅顯示邏輯上的配送順序，還展示各段的實際途經城市，幫助司機了解具體行駛路線。

4-2 系統限制

- 規模限制：TSP 的精確演算法（分支定界法和動態規劃法）在目的地數量增加時，計算時間會急劇上升。DP 的空間複雜度 $O(2^k * k)$ 限制了可處理的目的地數量（建議 $k \leq 20$ ）。
- 單車輛模型：系統目前只考慮單一車輛的配送規劃，未處理多車輛的車輛路徑問題 (VRP)。
- 未考慮實際因素：如道路交通狀況、配送時間窗口、車輛載重限制等實際運營因素。
- 資料不持久化：使用者建立的地圖在重新載入頁面後會消失，目前未支援儲存/匯入功能。

4-3 結論與反思

4-3.1 系統成果

本系統成功整合前端 HTML5 Canvas 渲染引擎與後端兩階段圖論優化引擎，使用者能流暢地在畫布上即時繪製、縮放自訂的城市拓撲地圖。當設定好起點與多個目的地並執行計算後，系統能排出最佳巡迴配送順序，並在結果區詳細拆解各路段的實際繞行軌跡與總耗時（例如載入 7 點範例地圖，系統能正確算得最短時間為 21 分鐘），在地圖上以動態物流車動畫直觀演示行駛路

線，系統具備防呆機制，當目的地數量過多時會自動切換為高效率的貪婪演算法以防止運算卡頓，且在開發過程中透過 AI 協同除錯，成功解決了畫布動畫重繪卡頓與動態修改道路權重時的資料同步瓶頸。

4-3.2 未來改進的方向

- 擴展為多車輛路徑問題 (VRP)：支援多輛車同時配送，分配目的地給不同車輛。
- 導入即時交通數據：透過 Google Maps API 等服務取得即時行駛時間。
- 支援時間窗口限制：考慮每個目的地的配送時間窗口。
- 使用更先進的啟發式演算法：如 2-opt、模擬退火 (Simulated Annealing) 或遺傳演算法 (Genetic Algorithm) 來處理大規模問題。
- 地圖資料持久化：支援將使用者建立的地圖匯出為 JSON 檔案，以及從 JSON 檔案匯入，實現資料的保存與分享。

4-3.3 遇到的挑戰

在開發過程中，主要遇到以下挑戰：

- 分支定界法的下界設計：一個好的下界函數能大幅減少搜尋空間。初始版本使用過於鬆散的下界（僅比較當前成本），修剪效果不佳。後來改進

為考慮所有未訪問節點的最短出邊之和，顯著提升了修剪效率。

- Canvas 動畫的平滑度：在繪製配送路線動畫時，需要計算每一幀的車輛位置。透過將總路徑按線段比例分配動畫進度，實現了流暢的動畫效果。
- 路徑還原：TSP 計算的是邏輯上的訪問順序，但實際展示需要顯示完整的物理路徑（途經哪些中繼城市）。透過 Dijkstra 的前驅節點 (prev) 陣列回溯重建完整路徑。
- 動態圖形編輯的座標轉換：實作畫布的平移和縮放功能時，需要正確地將螢幕座標轉換為世界座標 (screenToWorld) 和反向轉換 (worldToScreen)，以確保使用者在任何縮放和平移狀態下都能正確地點擊和拖曳節點。